

Mount Kenya University



Unlocking Infinite Possibilities

NAME : STEPHEN KIMANI

ADMISSION: BSCCS/2024/57444

SCHOOL OF COMPUTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

UNIT CODE: BIT 3208

UNIT TITLE : ADVANCED WEB DESIGN

**PROJECT TITLE : CAMPUS EVENT BOARD : A CAMPUS EVENT
MANAGEMENT SYSTEM**

Lecturer: Michael Nyoro

JUNE 2026

Contents

1	WEEK 1: LOCAL DEVELOPMENT ENVIRONMENT SETUP	4
1.1	Installation of XAMPP	4
1.2	Starting Apache and MySQL Service	4
1.2.1	Reflection	5
2	WEEK 2 : WIREFRAMES, GUI DESIGN AND PROJECT PLANNING	6
2.1	Homepage Wireframe.....	6
2.2	Student Login Page Wireframe	7
2.3	Admin Dashboard Wireframe	7
2.4	Graphical User Interface Design	8
2.5	Project workflow for Campus Event Board.....	8
2.5.1	Technology stack used in the development of the system.....	9
		9
2.5.2	Reflection	10
3	Week 3: JavaScript and PHP Basics	11
3.1	Event Product Catalog	11
3.2	Form Validation.....	11
3.3	Password Strength Checker	12
3.4	DOM Manipulation	12
	Figure 10: DOM manipulation using JavaScript to interact with event content.	12
3.5	PHP Syntax Implementation.....	13
3.5.1	Reflection	13
4	Week 4 : SERVER-SIDE PROCESSING AND BACKEND DEVELOPMENT	14
4.1	Folder Organization and Project Structure	14
4.2	User Authentication Records	15
4.3	PHP Form Processing	16
4.4	Database Connection Configuration.....	17
4.5	GitHub Repository and Version Control.....	18
4.6	Weekly Reflection	18

5	WEEK 5: Database Integration and CRUD Operations	19
5.1	Database Creation	19
5.2	Database Tables	20
5.3	Create Operation	21
5.4	Read Operation.....	22
5.5	Update Operation.....	23
5.6	Delete Operation.....	23
5.7	SQL Query Execution.....	24
5.8	User Authentication Records	24
5.9	GitHub Repository Update	25
5.9.1	Reflection	25

1 WEEK 1: LOCAL DEVELOPMENT ENVIRONMENT SETUP

1.1 Installation of XAMPP

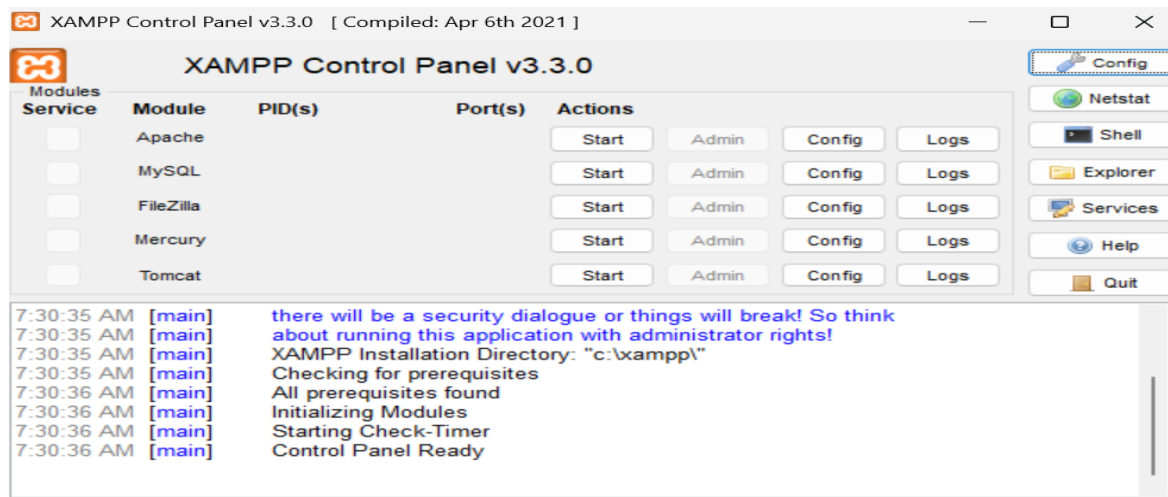


Figure 1: installation of XAMPP

1.2 Starting Apache and MySQL Service

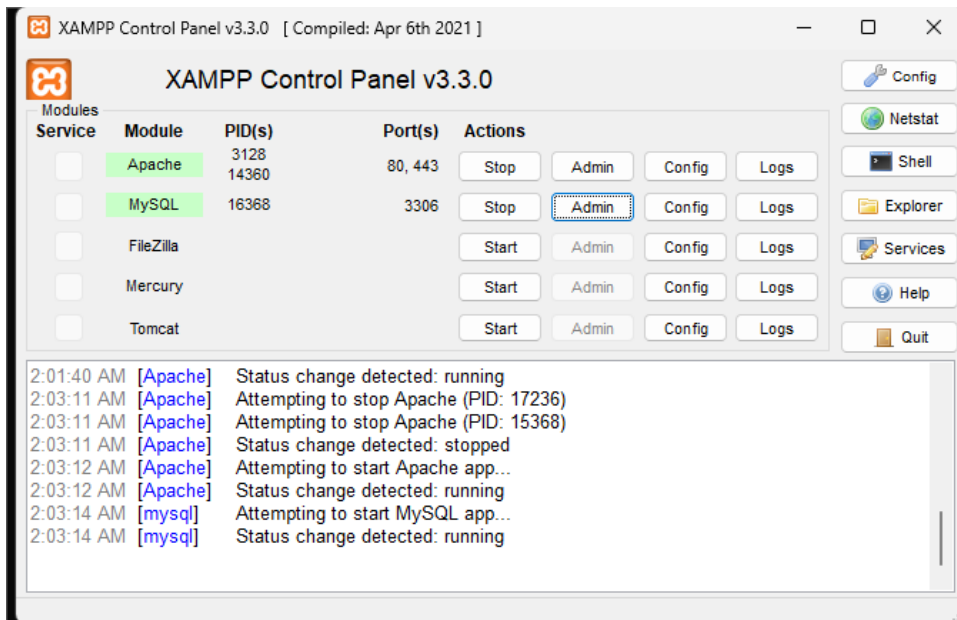


Figure 2: Apache and MySQL running

1.2.1 Reflection

During Week 1, I successfully set up XAMPP and configured Apache and MySQL for local web development. I learned how localhost works and how PHP applications communicate with a MySQL database. The main challenge was understanding project deployment within the htdocs directory.

2 WEEK 2: WIREFRAMES, GUI DESIGN AND PROJECT PLANNING

2.1 Homepage Wireframe

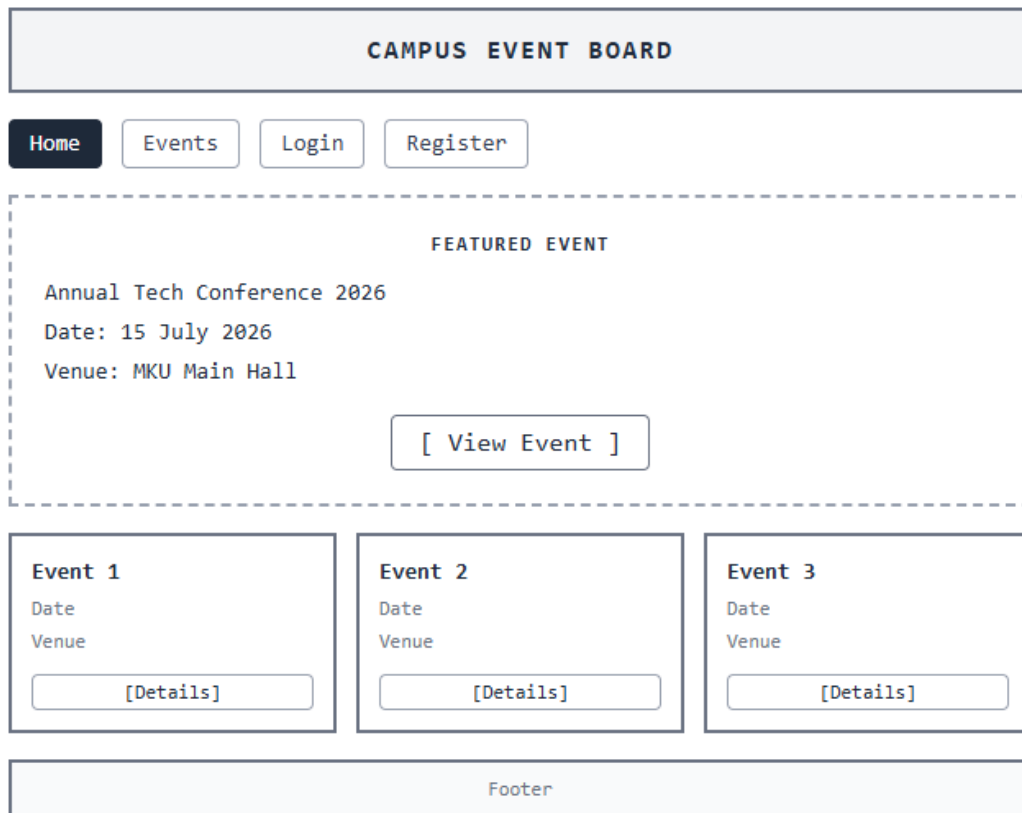


Figure 3:Homepage Wireframe for the Campus Event Board system.

2.2 Student Login Page Wireframe

CAMPUS EVENT BOARD

Student Login

Email

Password

[Login]

Don't have an account?

[Register]

Footer

Figure 4: Student Login Page Wireframe.

2.3 Admin Dashboard Wireframe

ADMIN DASHBOARD

Dashboard Events RSVPs Logout

12
Total Events

348
Total Students

521
Total RSVPs

Event Name	Date	Venue	Edit	Delete
Tech Day	...	...	[E]	[D]
Career Day	...	...	[E]	[D]

[Create New Event]

Figure 5: Admin Dashboard Wireframe.

2.4 Graphical User Interface Design

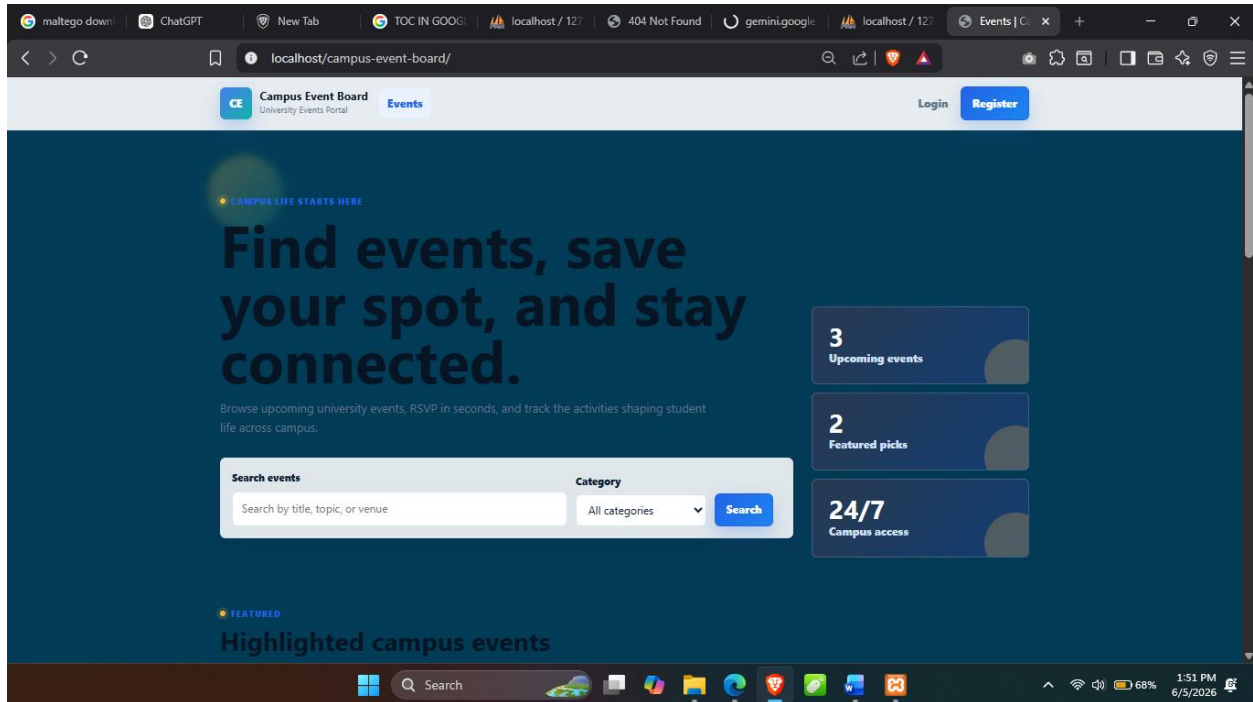
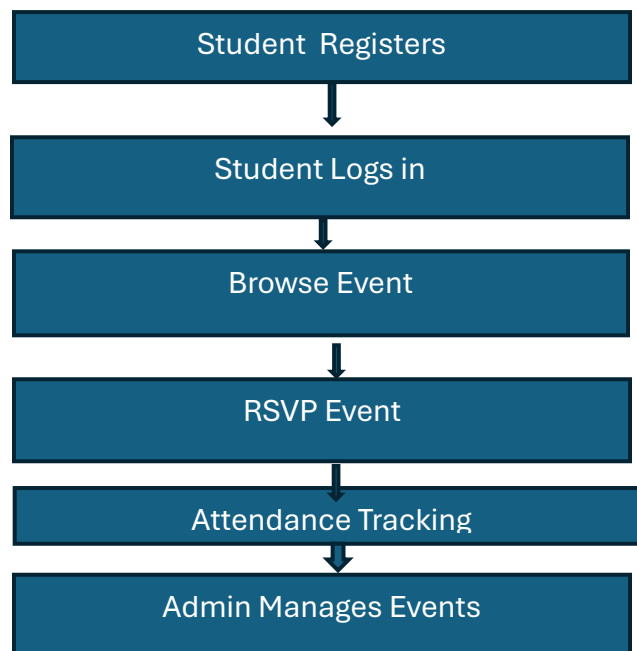
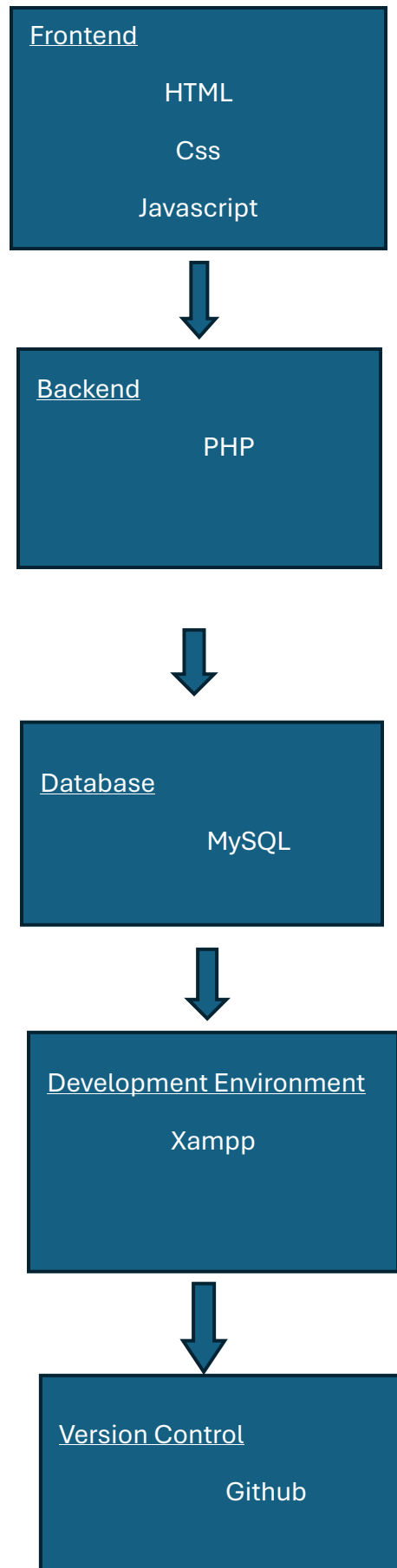


Figure 6: shows the Graphics User Interface of the campus event management system .

2.5 Project workflow for Campus Event Board.



2.5.1 Technology stack used in the development of the system



2.5.2 Reflection

During Week 2, I designed wireframes, GUI layouts, and a workflow for the Campus Event Board system. This helped me visualize system functionality before development. I also identified the technologies required for implementation, including HTML, CSS, JavaScript, PHP, and MySQL.

3 Week 3: JavaScript and PHP Basics

3.1 Event Product Catalog

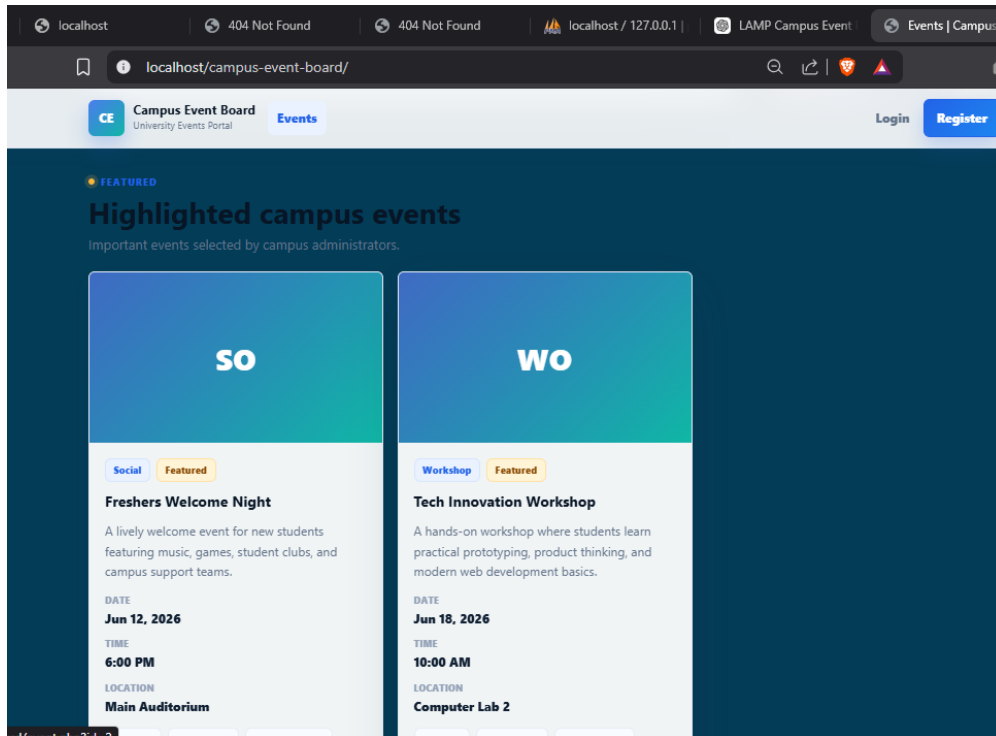


Figure 7: Event catalog developed using HTML, CSS and JavaScript.

3.2 Form Validation

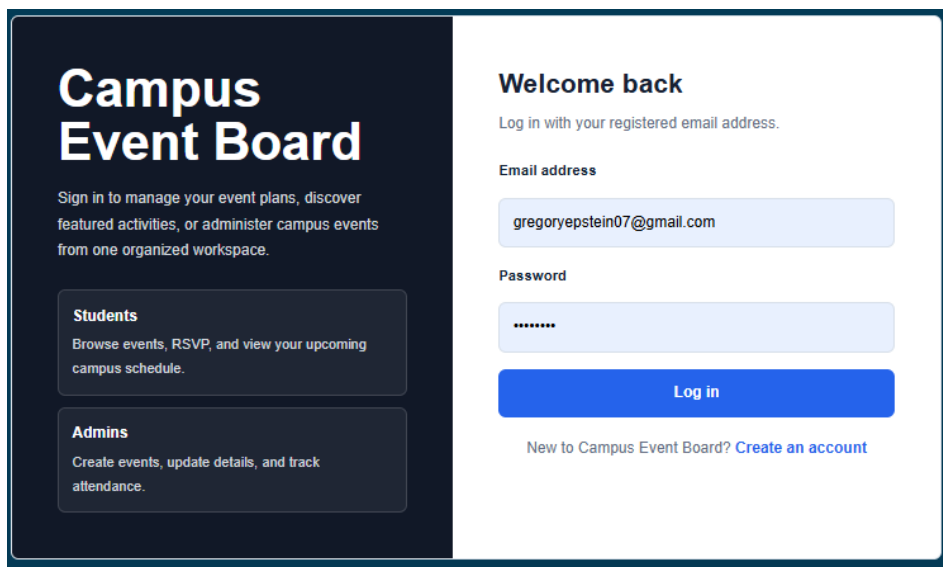


Figure 8: Login form validation implementation.

3.3 Password Strength Checker

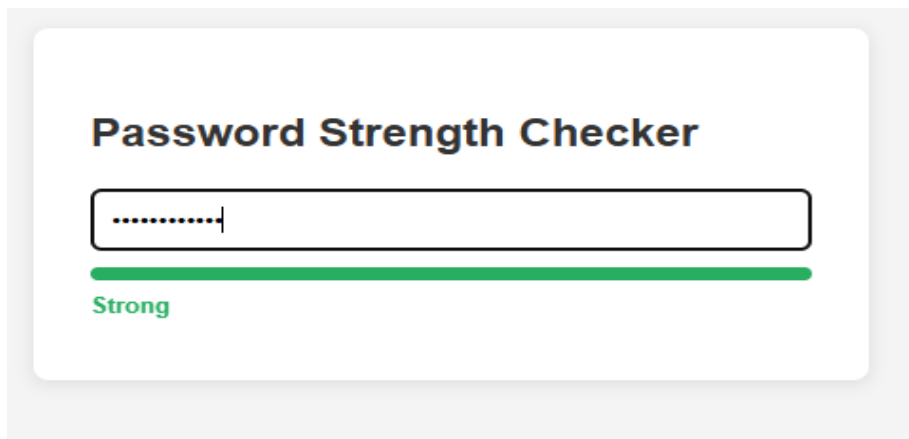


Figure 9: Password Strength Checker implementation

3.4 DOM Manipulation

```
SourceCode > assets > js > JS app.js > ...
1  document.addEventListener('DOMContentLoaded', () => {
2      initMobileNavigation();
3      initLiveEventSearch();
4      initConfirmations();
5      initImagePreview();
6      initFormValidation();
7      initModals();
8  });
9
10 /* Full JavaScript saved in your project file */
```

Figure 10: DOM manipulation using JavaScript to interact with event content.

3.5 PHP Syntax Implementation

```
SourceCode > event.php > section.content-section.event-info-section
1  <?php
2  declare(strict_types=1);
3
4  require_once __DIR__ . '/config/db.php';
5  require_once __DIR__ . '/includes/auth.php';
6
7  $eventId = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
8
9  if (!$eventId) {
10     header('Location: index.php');
11     exit;
12 }
13
14 $eventStatement = $pdo->prepare(
15     'SELECT
16         e.*,
17         COUNT(DISTINCT CASE WHEN r.status = "going" THEN r.id END) AS rsvp_count,
18         COUNT(DISTINCT a.id) AS attendance_count
19     FROM events e
20     LEFT JOIN rsmps r ON r.event_id = e.id
21     LEFT JOIN attendance a ON a.rsvp_id = r.id
22     WHERE e.id = :event_id
23     GROUP BY e.id
24     LIMIT 1'
25 );
26 $eventStatement->execute([
27     ':event_id' => $eventId,
28 ]);
29 $event = $eventStatement->fetch();
30
31 if (!$event) {
32     header('Location: index.php');
```

Figure 11: PHP syntax implementation for dynamic content generation.

3.5.1 Reflection

During Week 3, I implemented JavaScript and PHP features including an event catalog, form validation, password strength checking, and DOM manipulation. I gained practical experience in creating interactive web pages and using PHP to generate dynamic content for web applications.

4 Week 4: SERVER-SIDE PROCESSING AND BACKEND DEVELOPMENT

4.1 Folder Organization and Project Structure

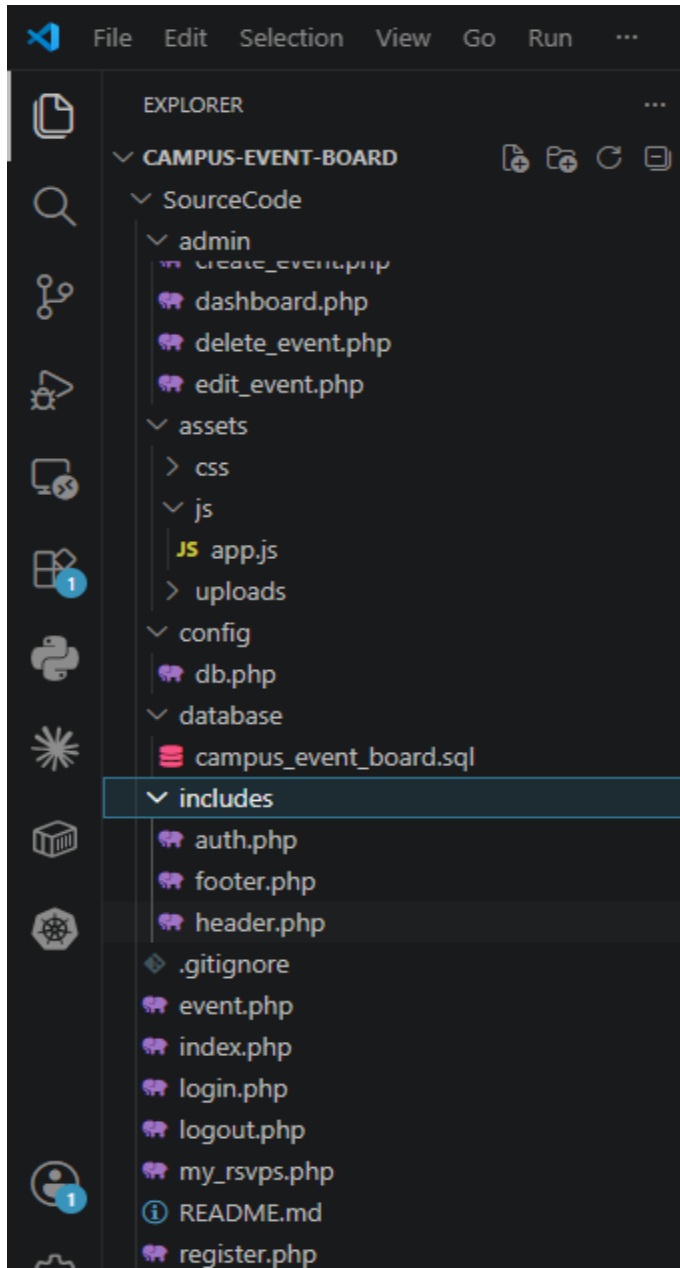
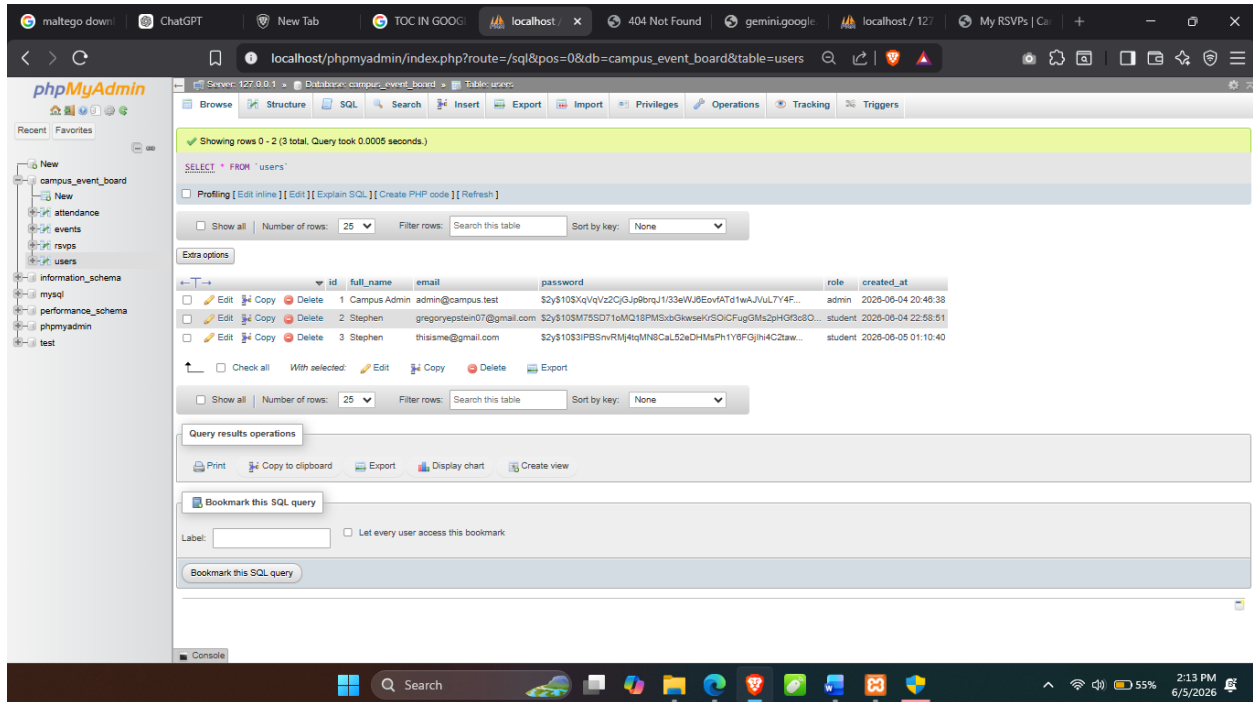


Figure 12: Folder organization and project structure for the Campus Event Board system.

4.2 User Authentication Records



The screenshot displays the phpMyAdmin interface for a MySQL database named 'campus_event_board'. The 'users' table is selected, and the query results show three rows of user authentication records. The table structure includes columns for id, full_name, email, password, role, and created_at. The data shows an admin user and two student users.

	id	full_name	email	password	role	created_at
☐	1	Campus Admin	admin@campus.test	\$2y\$10\$KqVqV2CjGj9RbqJ1/33eWJ9EovfATd1eAJuLTy4F...	admin	2020-06-04 20:48:38
☐	2	Stephen	gregoryepsten07@gmail.com	\$2y\$10\$M75SD71oMQ18PM5ibGkweK/SOICFugOMs2pHOF3c8O...	student	2020-06-04 22:58:51
☐	3	Stephen	thisisme@gmail.com	\$2y\$10\$3IPB5nRMj4tqMNBcaL52eCHMsPhY19F0j/h4C2aw...	student	2020-06-05 01:10:40

Figure 13: User authentication records stored in the MySQL database.

4.3 PHP Form Processing

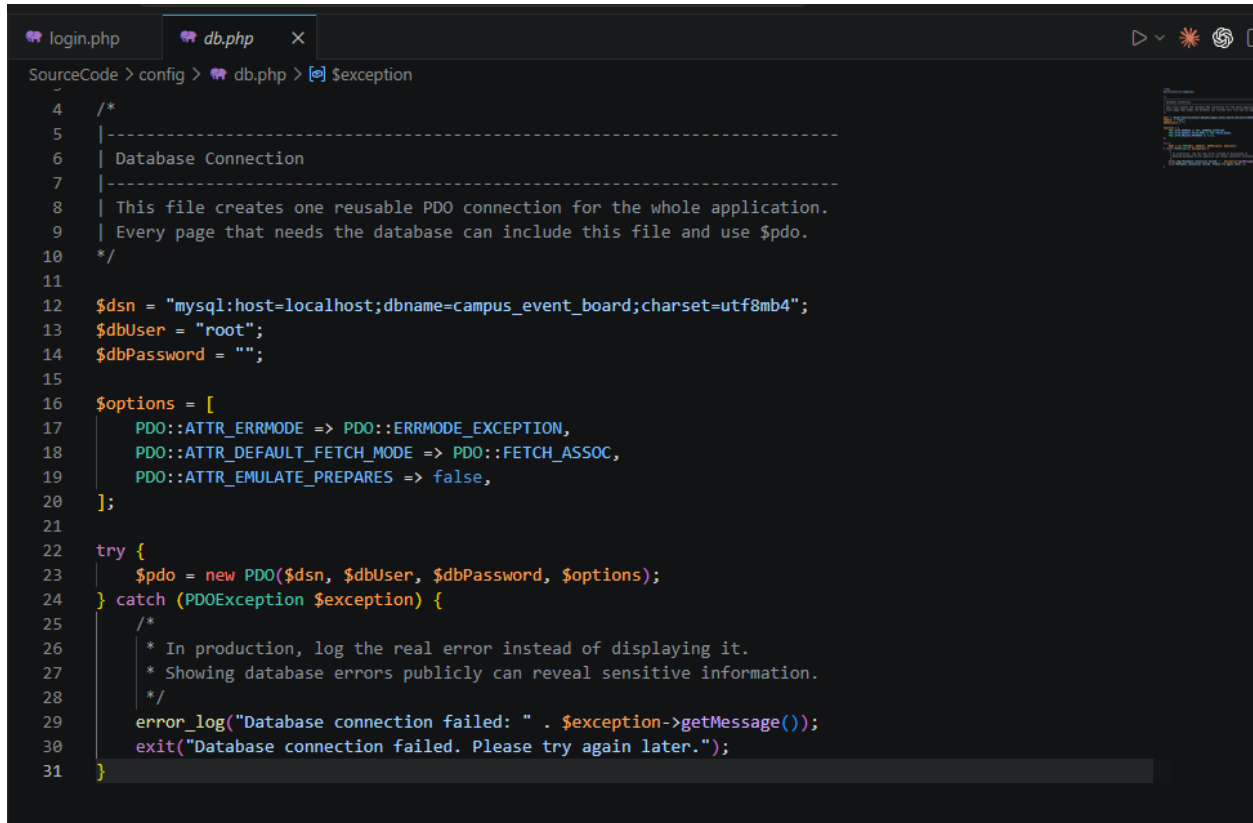
```
SourceCode > login.php > html
12
13     header('Location: index.php');
14     exit;
15 }
16
17 $email = '';
18 $errors = [];
19
20 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
21     $email = trim($_POST['email'] ?? '');
22     $password = $_POST['password'] ?? '';
23
24     /*
25      * Keep login errors general. This avoids telling attackers whether an
26      * email address exists in the system.
27      */
28     $loginError = 'Invalid email or password.';
29
30     if ($email === '' || $password === '') {
31         $errors[] = 'Email and password are required.';
32     } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
33         $errors[] = $loginError;
34     } else {
35         $userStatement = $pdo->prepare(
36             'SELECT id, full_name, email, password, role
37             FROM users
38             WHERE email = :email
39             LIMIT 1'
40         );
41         $userStatement->execute([
42             ':email' => $email,
43         ]);
44     }
45 }
```

Stephen Kimani (6 days ago) Ln 357, Col 8 Spaces: 4 UTF-8 CRLF PHP Go Live Prettier

Search 1:45 PM 6/19/2026

Figure 14: PHP processing of user login requests using POST data.

4.4 Database Connection Configuration



```
login.php db.php x
SourceCode > config > db.php > $exception
4  /*
5  |-----
6  | Database Connection
7  |-----
8  | This file creates one reusable PDO connection for the whole application.
9  | Every page that needs the database can include this file and use $pdo.
10 */
11
12 $dsn = "mysql:host=localhost;dbname=campus_event_board;charset=utf8mb4";
13 $dbUser = "root";
14 $dbPassword = "";
15
16 $options = [
17     PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
18     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
19     PDO::ATTR_EMULATE_PREPARES => false,
20 ];
21
22 try {
23     $pdo = new PDO($dsn, $dbUser, $dbPassword, $options);
24 } catch (PDOException $exception) {
25     /*
26     * In production, log the real error instead of displaying it.
27     * Showing database errors publicly can reveal sensitive information.
28     */
29     error_log("Database connection failed: " . $exception->getMessage());
30     exit("Database connection failed. Please try again later.");
31 }
```

Figure 15: Database connection configuration using PDO.

4.5 GitHub Repository and Version Control

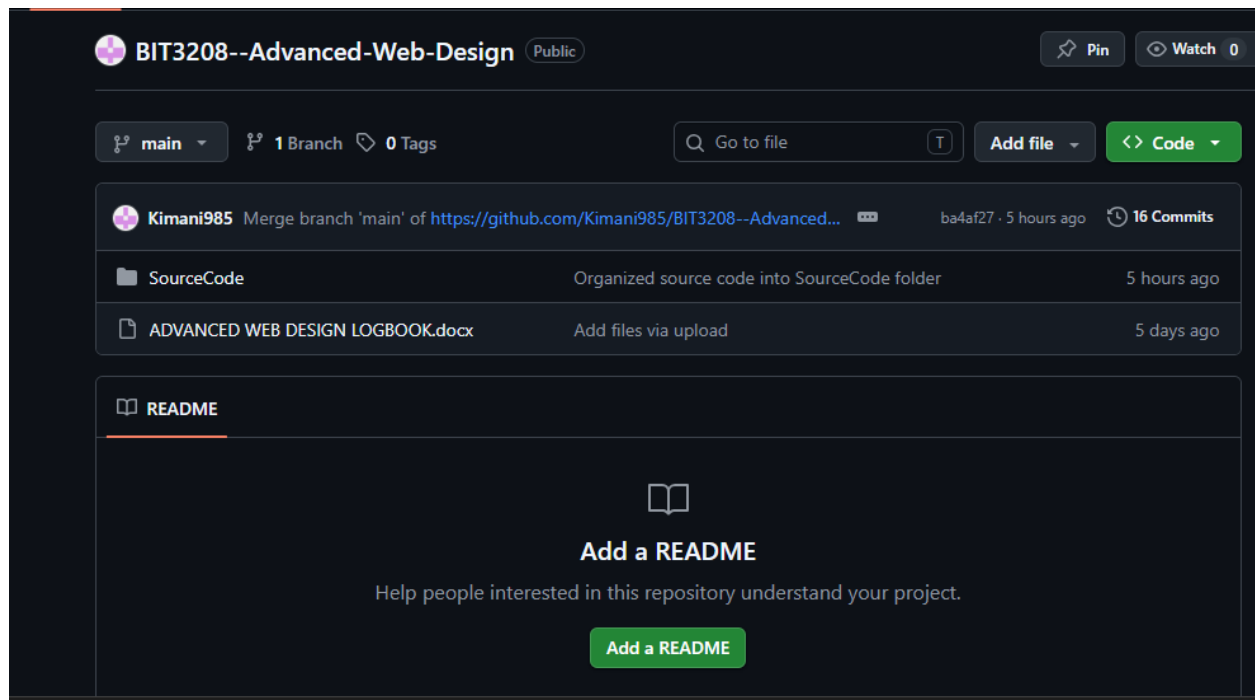


Figure 16: GitHub repository showing project version control.

4.6 Weekly Reflection

During Week 4, I learned how server-side processing works by integrating PHP with HTML forms and a MySQL database. I implemented login functionality, database connectivity using PDO, and user authentication mechanisms. I also organized the project structure professionally and used GitHub for version control and project backup.

5 WEEK 5: Database Integration and CRUD Operations

5.1 Database Creation

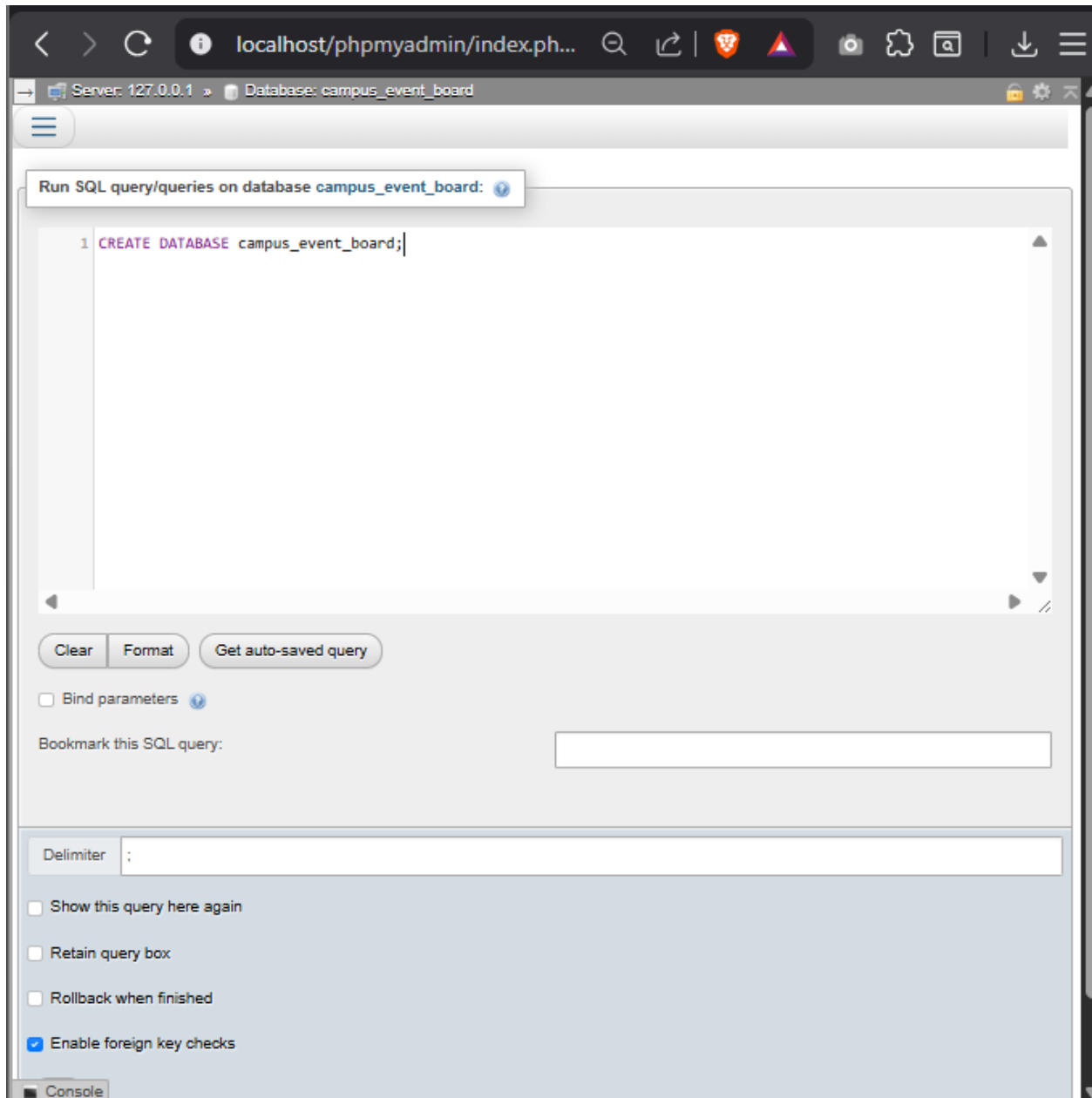
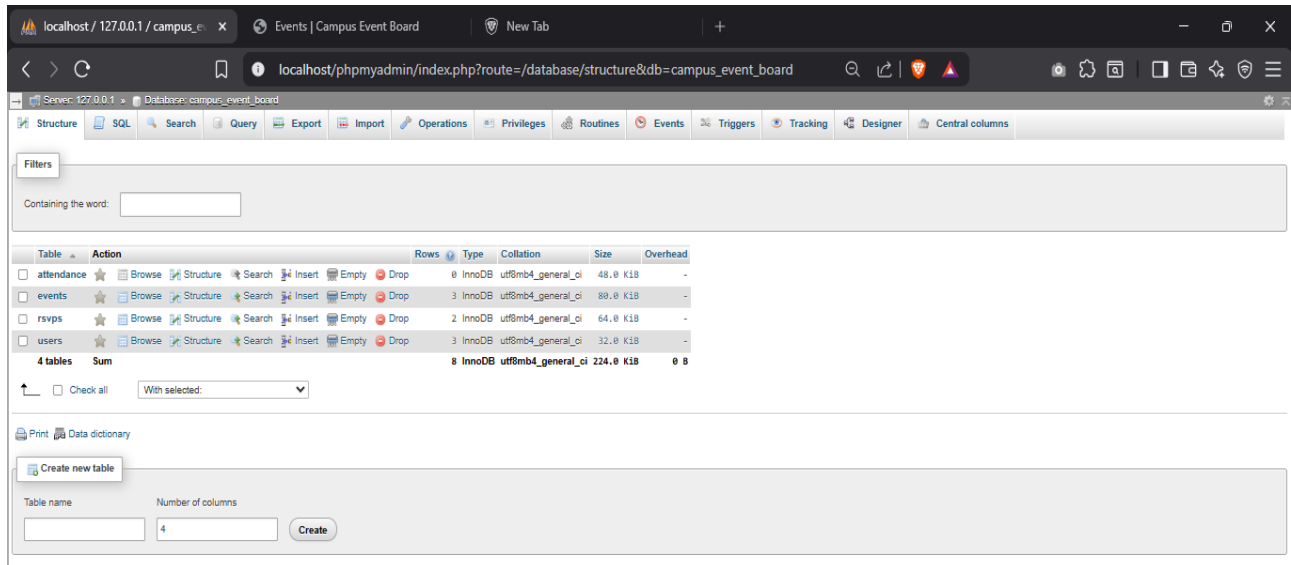


Figure 17: Creation of the Campus Event Board database using phpMyAdmin.

5.2 Database Tables



The screenshot shows the phpMyAdmin interface for a database named 'campus_event_board'. The 'Structure' tab is selected, displaying a table of database tables. The table has columns: Table, Action, Rows, Type, Collation, Size, and Overhead. The tables listed are 'attendance', 'events', 'rvsps', and 'users'. Below the table, there is a 'Create new table' button and a form to create a new table with fields for 'Table name' and 'Number of columns'.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ attendance	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 K1B	-
☐ events	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	88.0 K1B	-
☐ rvsps	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	64.0 K1B	-
☐ users	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 K1B	-
4 tables	Sum	8	InnoDB	utf8mb4_general_ci	224.0 K1B	0 B

☐ Check all With selected:

[Print](#) [Data dictionary](#)

[Create new table](#)

Table name: Number of columns: [Create](#)

Figure 18: Database tables created for the Campus Event Board system.

5.3 Create Operation

The screenshot shows a web browser at the URL `localhost/campus-event-board/SourceCode/admin/create_event.php`. The page has a dark blue header with the 'Campus Event Board' logo and navigation links for 'Events', 'Dashboard', and 'Create Event'. A 'Campus Admin' dropdown menu and a 'Logout' button are also present. The main content area is titled 'Create event' with a 'Back to dashboard' button. Below the title is a subtitle: 'Add a new campus event with details, capacity, image, and featured placement.' The form itself is divided into several sections: 'Event title' with a text input containing 'Team1 Africa Community call 16 🚩 Road to Mini Hack'; 'Description' with a text area containing three paragraphs about 'Team1 Africa Community Calls'; 'Category' with a dropdown set to 'TECH'; 'Venue' with a text input set to 'EVEREST HALL'; 'Date' with a date picker set to '07/18/2026'; 'Time' with a time picker set to '09:00 AM'; 'Capacity' with a text input set to '1000'; and 'Event image' with a 'Choose File' button and the filename 'ETHICAL HACKING COINFRENCE.png'. A note at the bottom right of the form states 'Optional .JPG, .PNG, or WEBP. Maximum 2MB.'

Figure 19: Create operation used to add a new event to the Campus Event Board system.

5.4 Read Operation

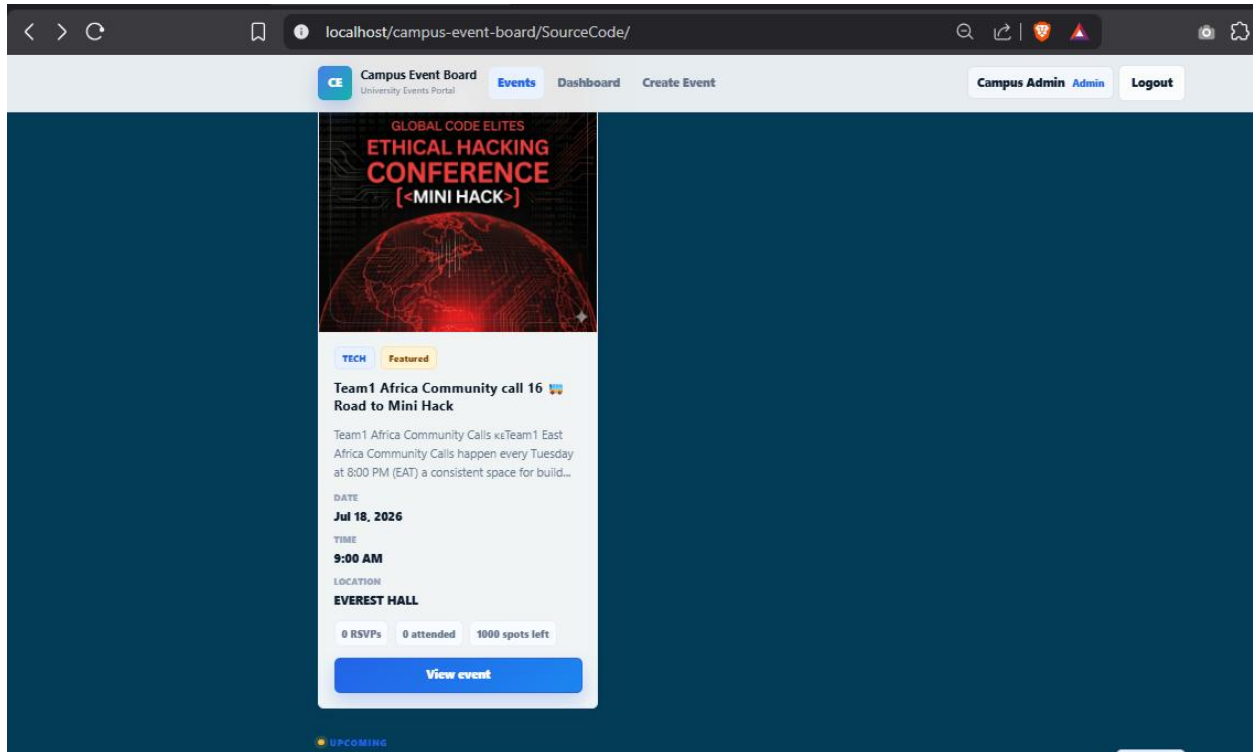


Figure 20: Read operation displaying events stored in the database.

5.5 Update Operation

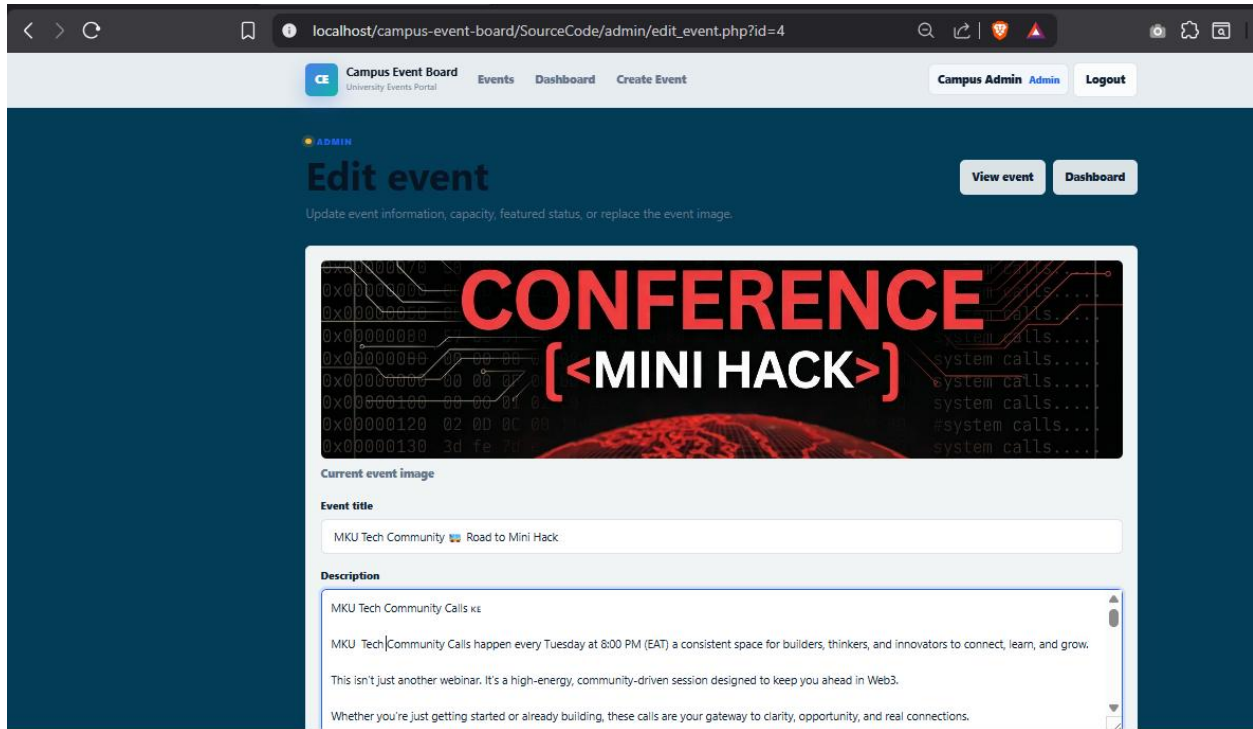


Figure 21: Update operation used to modify event information.

5.6 Delete Operation

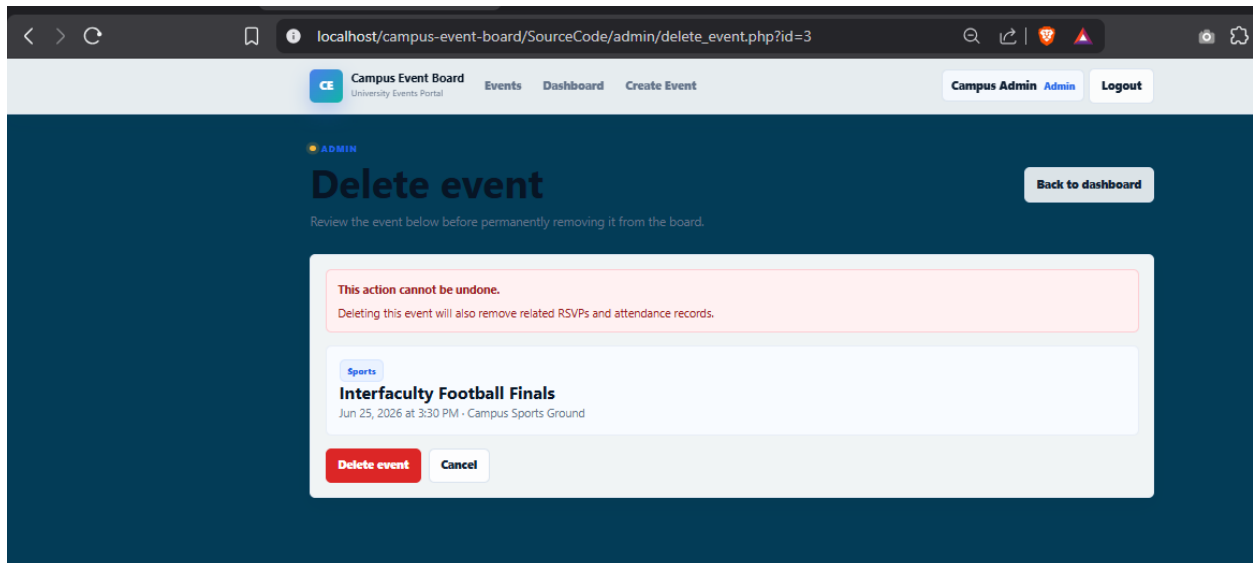


Figure 22: Delete operation available for removing event records from the system.

5.7 SQL Query Execution

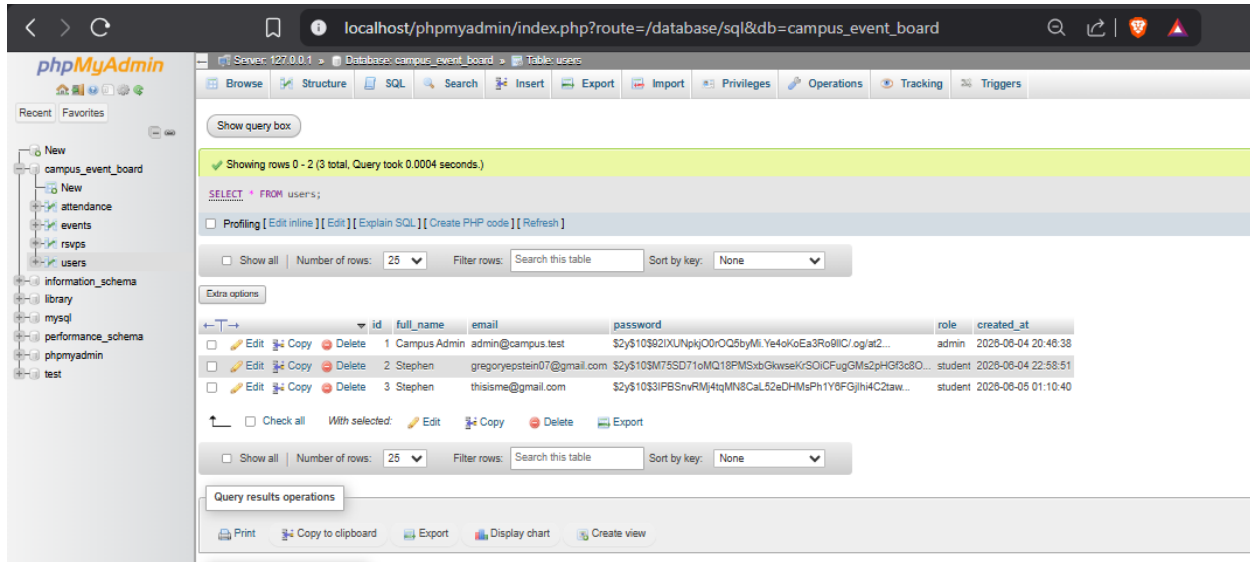


Figure 23: SQL query execution within phpMyAdmin.

5.8 User Authentication Records

Extra options						
	id	full_name	email	password	role	created_at
☐	1	Campus Admin	admin@campus.test	\$2y\$10\$92IXUNpkjO0rQ5byMi.Ye4oKoEa3Ro9llC/.og/at2...	admin	2026-06-04 20:46:38
☐	2	Stephen	gregoryepstein07@gmail.com	\$2y\$10\$M75SD71oMQ18PMSxbGkwseKrSOiCFugGMs2pHGf3c8O...	student	2026-06-04 22:58:51
☐	3	Stephen	thisisme@gmail.com	\$2y\$10\$3IPBSnvRMj4tqMN8CaL52eDHMsPh1Y8FGjih4C2taw...	student	2026-06-05 01:10:40

Figure 24: User authentication records stored in the Campus Event Board database.

5.9 GitHub Repository Update

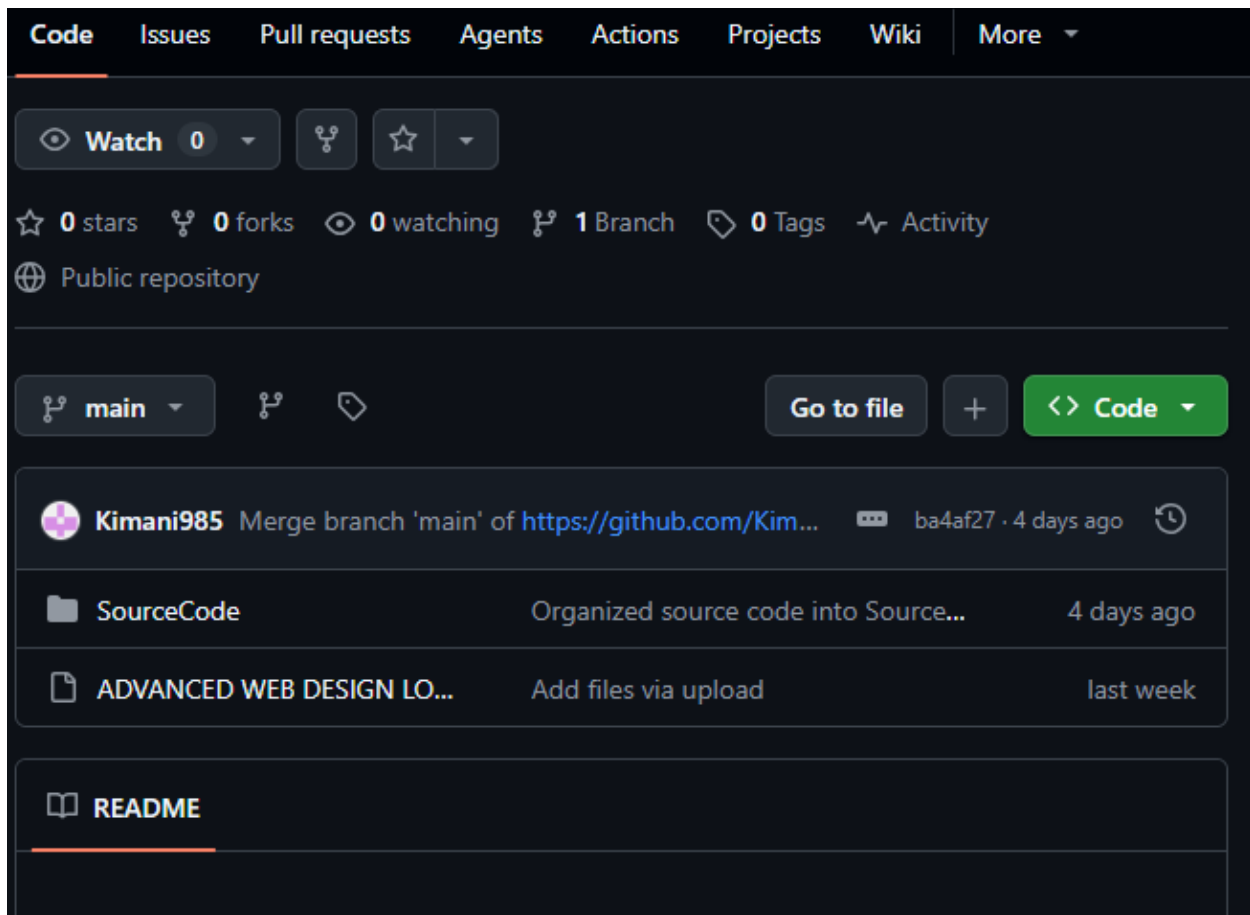


Figure 25: GitHub repository showing project source code, documentation, logbook, and creative implementation files.

5.9.1 Reflection

During Week 5, I integrated the Campus Event Board system with a MySQL database using PDO. CRUD operations were implemented and tested, enabling event creation, retrieval, modification, and deletion. SQL queries were executed through phpMyAdmin, and GitHub was used to manage and update project files.